刘谦

AI 应用 / AI 全栈

6 年工作经验 | 17317619053 | tunshisky@outlook.com | 本科 | 32 岁

核心能力：前端架构 / 低代码与配置驱动 / 数据密集型页面与可视化 / 全栈与接口协作 / AI 工程化落地

职业概述

具备 SaaS 数据平台与多端企业应用开发经验，擅长复杂表单、权限流程、低代码配置、数据可视化及监控告警等场景，能够独立推进需求分析、方案设计、开发联调与上线交付。具备 Node.js、NestJS、Prisma 和 MySQL 实战经验，可完成数据建模、接口开发与鉴权设计；结合 RAG、工具调用、多 Agent 编排及 SSE 流式交互实践，能够贯通前端交互、后端服务、数据存储与 AI 能力集成，完成业务应用的开发、验证及持续迭代。

专业技能

AI 应用开发

- **AI 应用架构：**LangChain.js / LangGraph，Agent 设计（ReAct、FSM、多智能体协作），Function Calling 与工具编排，Agent 健壮性（超时、重试、降级、幂等等 8 类企业级失败场景的处理）
- **RAG 知识库：**文档切分与清洗，Embedding，pgvector 向量检索，召回与重排优化
- **Agent 与工作流编排：**使用 LangGraph StateGraph、ToolNode 构建 ReAct 与多 Agent 工作流，实现意图路由、工具串联、状态传递和结果汇总；结合 checkpoint、thread_id 实现流程挂起与恢复。
- **评估与调优：**固定测试输入与模型参数，通过多轮采样比较回答质量、工具执行结果、P95 延迟及 Token 消耗，为 Prompt、检索策略和模型选择提供依据。

智能体协作与 AI 工程化

- 熟练使用 Claude Code、Cursor、Copilot 等 AI 编程工具，具备 Prompt 工程能力，建立 AI 辅助开发规范体系，开发自定义 Skill 指令与 MCP 服务器
- 使用 SpecKit 规范化开发流程，让 AI 深度参与开发全流程，开发效率大幅提升。
- 采用目标与验收驱动的 AI 研发方式，先明确业务目标、功能边界、接口契约和完成标准，再让智能体参与方案分析与实现，关键规则和技术决策由本人负责。
- AI 生成代码必须经过人工 Diff 审查、类型检查、自动化测试和生产构建，并把高频问题回写到规则与上下文中，形成需求、实现、验证与知识沉淀闭环。

Node.js / NestJS / Python 全栈能力

- 使用 Node.js、NestJS、Prisma 和 MySQL 开发实际业务接口，按业务模块拆分 Controller、Service、DTO 和数据访问逻辑。能够设计 REST API、数据库模型及关联关系，完成 DTO 参数校验、分页筛选、统一响应、异常处理和 Swagger 接口文档。
- 具备基于 JWT 的 Access Token、Refresh Token、HttpOnly Cookie 和接口守卫实践，能够处理登录注册、令牌刷新、退出及受保护接口访问。
- 能够完成 React、TanStack Query 与 NestJS 接口联调，并使用 Vitest 对 JWT 鉴权、DTO、业务规则和权限边界进行测试。
- **数据库：**PostgreSQL / pgvector，MySQL，Redis
- **工程化与部署：**Docker，CI / CD，前端 AI 工程化（流式渲染、Function Calling 可视化、性能优化）

团队级 AI 提效落地能力

- **设计团队级 AI 提效落地路线：**根据团队现状（规模、技术栈、成熟度），制定包含基础设施搭建、试点项目验证、全面推广、规模化自动化的四阶段落地计划，并预估各阶段的时间投入与预期收益。
- **主导 Spec 驱动开发的团队实施：**组织团队编写 skill.md（团队规则），建立可复用的规范模板库（组件规范、API 规范、测试规范），设计 AI 代码审查的质量门禁机制（测试覆盖率、lint 通过率、规范一致性），保障 AI 生成代码的质量与一致性。
- **搭建团队 AI 工具链与自动化流水线：**组合使用 Trae、Codex、Claude、DeepSeek 等多工具，构建从设计稿到可部署代码的自动化流程；通过 MCP 连接内部组件库、设计系统与 API 文档；将 AI 能力封装为团队共享的 Skills，通过 /skills 命令实现一键复用。

工作经历

蓝想数科 | AI 应用

2025.06 — 至今

角色定位：负责预检分诊、急诊抢救、重症监护和麻醉随访等核心模块的完整交付，把复杂临床规则转化为稳定、可验证的业务流程；同时参与接口契约、监控数据与智能体研发规范建设。

技术栈：React、NestJs、Prisma、MySQL、LangChain、LangGraph、DeepAgent、Claude Code、Cursor、MCP、Skills、Rules

急诊重症 AI 智能客服 | 高可靠流程与可观测性

项目概述：急诊抢救具有时间敏感、角色协同密集和数据准确性要求高的特点。系统将分散的诊疗数据组织成连续业务流程，并通过自动化验证、监控埋点和日志排查保障高频迭代下的稳定性。

项目价值：把复杂页面建设提升为数据模型、状态流程、前后端契约、质量体系与监控链路的协同设计，形成“规则结构化—分层实现—监控反馈—自动化验证—人工验收”的交付方式。

工作职责

- 负责医疗 AI 应用服务架构设计与开发，围绕急诊、重症业务需求，建设模型接入、知识检索、Agent 编排及流式接口等核心模块。
- 负责医疗知识库建设与维护，围绕急诊处置规范、重症护理制度及医疗操作流程，实现文档处理、知识检索、智能问答及答案来源追溯。
- 负责医疗 AI Agent 工作流开发，基于 LangGraph 实现用户意图识别、条件路由、知识检索、工具调用及结果整合，支持工作流状态追踪与流式输出。
- 负责国内大模型及 Embedding 模型的接入与适配，支持 DeepSeek、智谱 AI、阿里云百炼等模型，开展模型效果、检索质量、响应耗时及调用成本评估。

上海领鸟云 | 全栈开发

2022.07 — 2025.06

角色定位：负责客户管理、数据分析、选址分析和产业图谱等 SaaS 数据平台模块，以及低代码配置平台的核心开发，从业务页面交付扩展到公共组件、可视化体系与配置平台建设。

技术栈：React、TypeScript、MySql、NestJs、Prisma、Docker、Jenkins

企业级数据智能与低代码分析平台 | 配置驱动的可视化体系

项目概述：企业数据分析场景持续扩展，传统“一项需求对应一套页面”的方式难以应对多数据源、多分析维度和频繁变化的需求。平台需要同时支撑数据查询、关系洞察和自助式页面搭建，并保证交互与数据表达的一致性。

工作职责

- 独立承担客户管理、数据分析、选址分析和产业图谱等模块的需求分析、技术方案与交付，在产品、后端及数据团队之间推进接口和交互方案落地。
- 将反复出现的查询、表格、字典和图表需求提升为公共能力，推动团队从逐页开发转向基于统一模式组合业务页面。
- 深度参与低代码工具建设，将页面结构、组件属性和数据关系抽象为配置模型，支撑编辑、保存、预览与运行时渲染的完整链路。
- 建立多类型图表的数据转换和接入方式，并参与 Jenkins 构建与多环境部署流程，兼顾业务交付效率、可视化一致性和发布稳定性。

上海联蔚数科 | 前端开发 2019.09 — 2022.07

角色定位：企业多端应用开发，完成从 React 页面实现到跨页面状态、多端适配和完整业务模块交付的能力积累。

技术栈：React、TypeScript、Redux、React Router、Ant Design、miniprogram、Taro、Cordova

- 负责微信小程序、H5 和 PC 管理后台的业务模块开发，从页面实现逐步承担接口联调、状态设计、平台适配和上线交付。
- 使用 Redux 管理跨页面共享状态，结合持久化与异步流程处理多步骤业务中的数据一致性，建立对复杂前端状态边界的系统认识。
- 将公共业务逻辑与小程序、H5 平台能力分层，集中处理授权、分享、分包和平台 API 差异，降低多端重复实现与维护成本。
- 围绕移动端体验实施路由懒加载、资源策略、请求合并和分包优化，形成兼顾功能交付、平台差异和性能体验的多端开发能力。

项目经验

蓝想数科 急诊重症临床协同平台

项目背景：面向急诊抢救、重症监护等临床协同场景，业务具有时间敏感、角色协作密集和数据准确性要求高的特点，需要将分散的诊疗数据组织为连续、可验证的业务流程，并保障高频迭代下的稳定性。

- **独立设计四层 AI 能力架构：**基础对话（SSE 流式输出）→ Function Calling 工具调用（订单查询、下单、退款等 4 个工具）→ RAG 知识库问答（pgvector 向量检索）→ LangGraph 多 Agent 工作流编排，实现意图识别自动路由
- 分别用 Node.js / TypeScript 与 Python / FastAPI 完整实现同一套业务架构，验证核心设计在不同技术栈下的可移植性，产出双语言对照的完整技术文档

技术栈：LangChain· LangGraph · RAG · pgvector · DeepSeek · React · NestJS / FastAPI

关键难点与方案

- **Agent 健壮性设计：**客服 Agent 在生产场景下会遇到工具调用超时、模型幻觉误报库存、连续调用同一工具陷入死循环、多轮对话导致上下文超长截断丢失关键订单信息、DeepSeek 服务临时不可用、用户中途打断对话等 8 类失败场景。逐一设计对应的兜底策略（超时熔断、失败结果显式回填给模型而非静默吞掉、最大轮次限制防死循环、历史消息裁剪 + Token 感知截断、Provider 中断自动降级提示），把单点故障对整体对话体验的影响控制在最小范围，而不是遇到异常直接抛 500 中断会话。

- **RAG 召回精度调优：**知识库问答早期出现"检索到片段但内容与问题弱相关，模型仍强行组织回答"导致的文不对题问题。通过反复测试 chunk_size / overlap 参数、引入相似度阈值过滤低相关片段、叠加防幻觉 Prompt 与数字一致性约束三层机制，将退款政策类问答的准确率提升到可直接面向用户使用的水平，避免了因为回答金额、天数等具体数字有误引发的客诉风险。
- **SSE 流式与工具调用中间态展示的一致性：**前端需要同时消费"逐字流式生成的文本"和"工具调用执行进度"两条并行的信息流，早期实现里两类事件互相插队导致界面渲染错乱（工具还没执行完文字已经开始流式输出）。重新设计统一的 SSE 事件协议（token / step / tool_result / done 分类型推送），前端按事件类型驱动独立的状态机，彻底解决了异步竞态导致的界面显示错误。

交付结果：形成结构化规则、分层实现、监控反馈、自动化验证和人工验收的交付流程，为高频迭代提供回归与问题追踪依据。

上海领鸟云　企业级数据智能与低代码分析平台

项目背景：面向多数据源、多分析维度及频繁变化的企业分析需求，支持数据查询、空间分析、关系洞察和自助式页面搭建。

技术栈：React、TypeScript、TanStack Query、Zustand、Ant Design、ECharts、Vite、Jenkins。

关键难点与方案

- **公共能力抽象：**不同业务模块重复实现查询、表格和详情编辑，复用时又容易引入业务耦合。将这些能力拆为可组合单元，通过输入输出契约明确边界，支持业务模块按需组装。
- **可视化适配层：**后端数据结构不统一，导致各页面重复处理数据转换，图表交互也难以保持一致。引入统一图表模型，再生成 ECharts 配置，集中处理联动、缩放、自适应与异常兜底。
- **配置驱动链路：**页面结构、组件属性和数据源关系需要共同保存，否则编辑、预览与实际运行容易出现差异。将三者统一描述为配置模型，贯通编辑、保存、预览和运行时渲染，并通过组件注册机制支持扩展。
- **缓存与交付：**这条混合了两个问题，建议拆开：用 TanStack Query 统一管理服务端数据缓存与失效；用 Vite 和 Jenkins 规范检查、构建及多环境发布流程。

交付结果：沉淀统一的数据接入、图表表达与页面搭建模型，将分散的业务页面开发转化为可复用的平台能力。

项目价值：将分散的数据页面开发提升为可复用的平台能力，使数据接入、图表表达和页面搭建拥有统一模型，体现复杂 ToB 系统中的前端架构与产品抽象能力。